Fresh + Deno × Angular + Spring

Desempenho de um quiz ao vivo com 1, 30, 60 e 100 jogadores

Fresh + Deno (SSR) **Angular + Spring (SPA)**

Coleta de 12/10/2026 · 24 execuções · mesma máquina, mesmo Nginx e PostgreSQL

Duas arquiteturas completas, não dois frameworks

Fresh + Deno (SSR)

1. **Nginx**
   encaminha todas as rotas ao Deno

2. **Deno · 1 processo**
   deno serve sem --parallel: HTML, API e SSE

3. **Navegador**
   recebe HTML pronto e hidrata ilhas

Angular + Spring (SPA)

1. **Nginx**
   serve o build estático; /api vai ao Spring

2. **Spring Boot (JVM)**
   API e SSE, multi-thread

3. **Navegador**
   baixa o bundle e renderiza

Iguais nas duas: Nginx, PostgreSQL, máquina e roteiro de carga.

Fresh + Deno (SSR) Angular + Spring (SPA)

Como medimos: 4 cargas, 3 pares cada, uma máquina

24
execuções no total

4
cargas: 1, 30, 60 e 100 jogadores

3
pares Fresh/Angular por carga

2
navegadores: host e 1 jogador

Caminho de cada requisição

Carga
clientes HTTP/SSE + 2 Chromium

→

Nginx
o mesmo nas duas

→

Servidor da aplicação
Deno ou **Spring Boot**

→

PostgreSQL
o mesmo nas duas

Limites: resultado exploratório, tudo na mesma máquina (Mac). 100 jogadores não são 100 navegadores: só host e 1 jogador usam Chromium.

Dicionário (1/2): arquitetura

SSR

Server-Side Rendering: o servidor monta o HTML e o navegador já recebe a página pronta.

SPA

Single-Page Application: o navegador baixa um app em JavaScript e monta as telas sozinho.

Hidratação e ilhas

O JavaScript liga a interatividade no HTML pronto. No Fresh, só nas partes interativas (ilhas).

Bundle

Pacote de JavaScript e CSS que a SPA baixa antes de mostrar a primeira tela.

Nginx

Servidor web na frente da aplicação: entrega arquivos e repassa requisições.

JVM

Máquina virtual Java onde roda o Spring Boot. Usa várias threads e reserva memória ao iniciar.

SSE

Server-Sent Events: conexão aberta em que o servidor envia eventos ao vivo ao navegador.

Processo único / event loop

O Deno executa o JavaScript num laço só (event loop): uma tarefa por vez, em 1 núcleo.

deno serve --parallel

Opção que sobe várias instâncias do Deno e divide as requisições entre núcleos. Não usada aqui.

Dicionário (2/2): medidas e carga

Carga, execução, par

Carga = nº de jogadores (1, 30, 60, 100). Execução = uma rodada de uma implementação. Par = uma execução de cada.

p95

Valor que 95 de cada 100 medições não ultrapassam. Ex.: p95 = 7,6 s → as 5 piores levaram mais.

Mediana

Valor do meio: metade das medições foi mais rápida, metade mais lenta.

RSS

Memória RAM que o processo ocupa de fato, em MiB.

CPU %

Uso do processador em % de 1 núcleo: 100% = um núcleo inteiro ocupado.

Núcleo e saturação

Núcleo executa uma tarefa por vez. Saturado = ocupado ~100% do tempo, sem folga.

Fila

Requisições que chegam com o núcleo ocupado esperam a vez; a espera soma no tempo de resposta.

Throttling

Limitar de propósito CPU ou rede do navegador para simular celular fraco ou 4G. Não usado aqui.

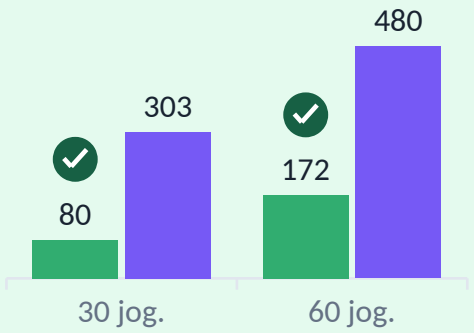
Fresh vence até 60; em 100, o Deno enfileira

1–60 jog. **Fresh** | 100 jog. **Angular**

1 a 60 jogadores

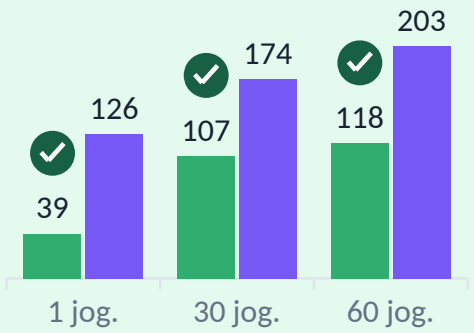
Fresh + Deno mais rápido e mais leve

p95 HTTP (ms)



95 de cada 100 respostas em até 172 ms no Fresh e 480 ms no Angular (60 jog.).

Memória RSS (MiB)

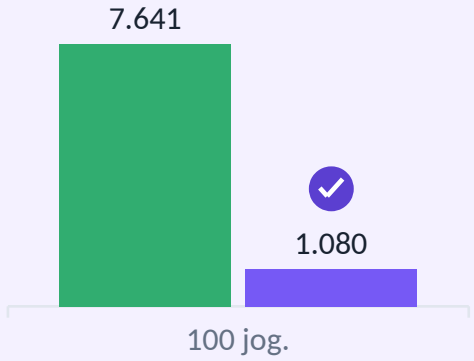


RAM ocupada pelo servidor: 118 contra 203 MiB com 60 jogadores.

100 jogadores

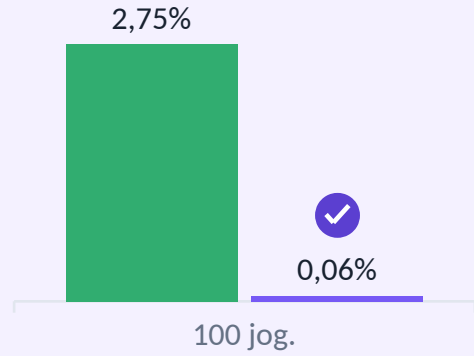
Deno no limite de 1 núcleo: pedidos esperam na fila

p95 HTTP (ms)



95 de cada 100 respostas do Fresh em até 7,6 s; as 5 piores levaram mais.

Respostas perdidas (%)



Fresh: 17 a 93 de 1.600 por execução. Angular: 1 de 1.600.

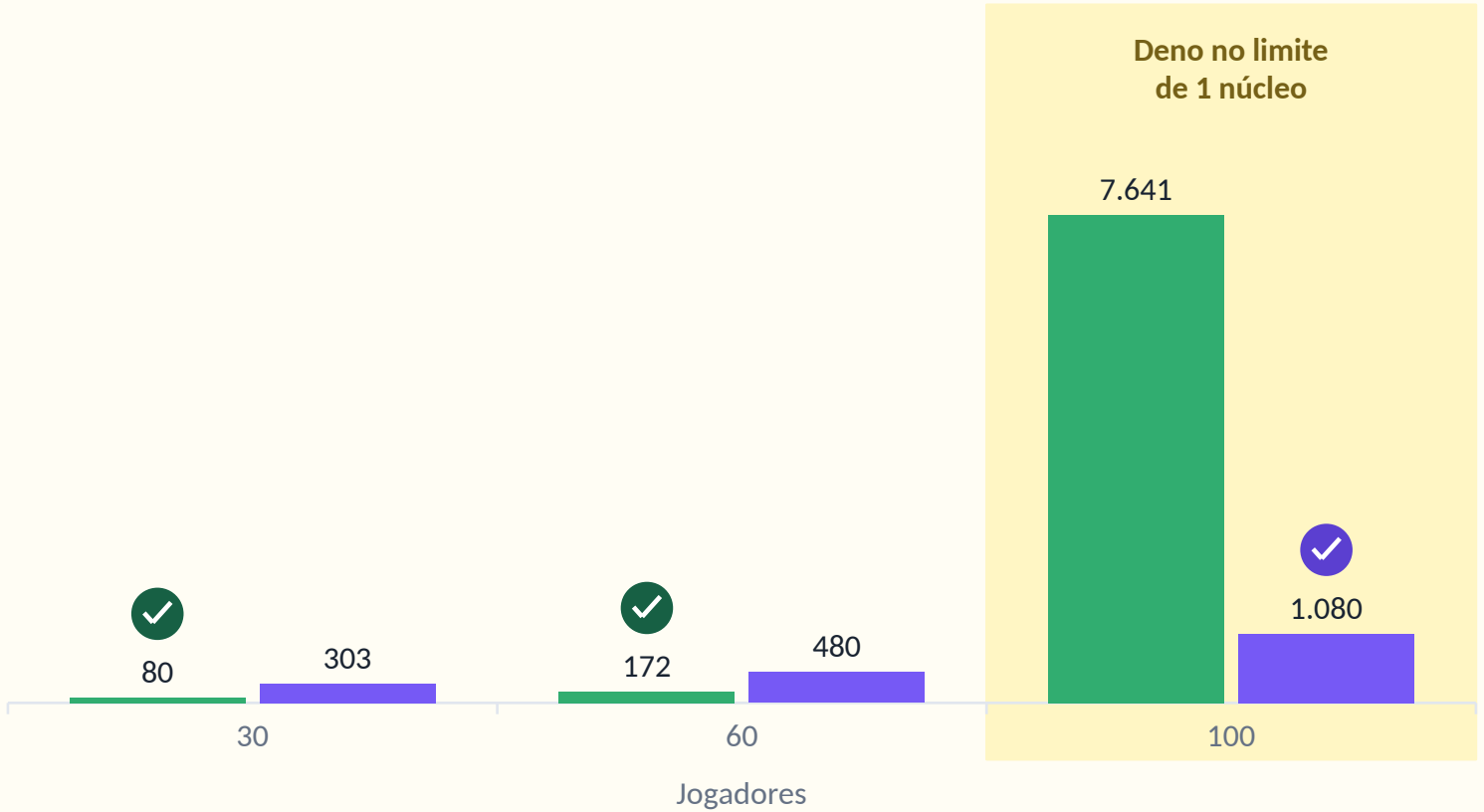
Fresh + Deno (SSR) | Angular + Spring (SPA) | vence nesta carga

Coleta de 12/10/2026 · 24 execuções · 3 pares por carga · mesma máquina

HTTP: Fresh ~3× melhor até 60, 7× pior em 100

30–60 jog. **Fresh** | 100 jog. **Angular**

p95 do tempo de resposta HTTP (ms) · *menor é melhor*



Faixa amarela: o Deno usa 1 núcleo inteiro e os pedidos novos esperam na fila (slide 8).

7,6 s

p95 HTTP do Fresh + Deno com 100 jogadores

Contra 1,1 s do Angular + Spring. Com 30 e 60 jogadores o Fresh respondeu em 80 e 172 ms.

Como ler o p95
95 de cada 100 respostas levaram até 7,6 s; as 5 piores, mais.

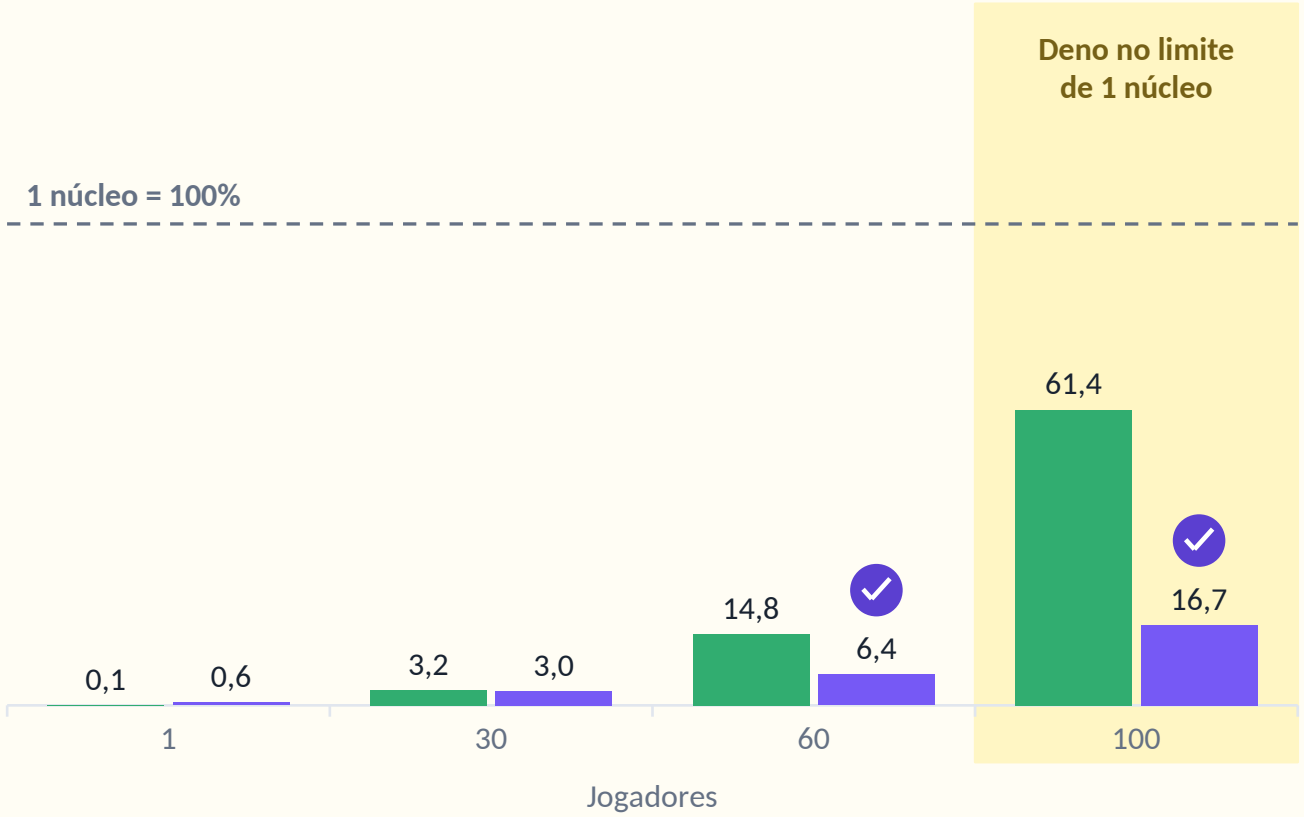
Fresh + Deno (SSR) · Angular + Spring (SPA) · vence nesta carga

Coleta de 12/10/2026 · 24 execuções · 3 pares por carga · mesma máquina

CPU: em 100, o Deno ocupa um núcleo inteiro

	1–30 jog.	60–100 jog.
★	Empate	Angular

CPU média do servidor (% de 1 núcleo) · *menor é melhor*



A média (61,4%) esconde os picos: p95 de CPU 102–103%, pico 108%.

O que é “no limite de 1 núcleo”

Chegam pedidos de 100 jogadores

↓

1 processo Deno roda o JavaScript em 1 núcleo, ~100% ocupado

↓

Pedidos novos esperam na fila

↓

p95 HTTP 7,6 s e 2,75% de respostas perdidas

Spring (multi-thread): pico de 80%, com folga.

Fresh + Deno (SSR) · Angular + Spring (SPA) · ✓ vence nesta carga

Coleta de 12/10/2026 · 24 execuções · 3 pares por carga · mesma máquina

SSE: misto até 60; em 100, Fresh atrasa e falha

1–60 jog. **Misto** | 100 jog. **Angular**

p95 da entrega de eventos SSE ao host (ms) · *menor é melhor*



Ex.: p95 = 5,6 s → 95 de cada 100 eventos chegaram ao host em até 5,6 s.

Respostas perdidas com 100 jogadores

2,75%

Fresh + Deno
17, 93 e 22 de 1.600 por execução

0,06%

Angular + Spring
1 de 1.600

Até 60 jogadores: nenhuma resposta perdida nas duas (empate).

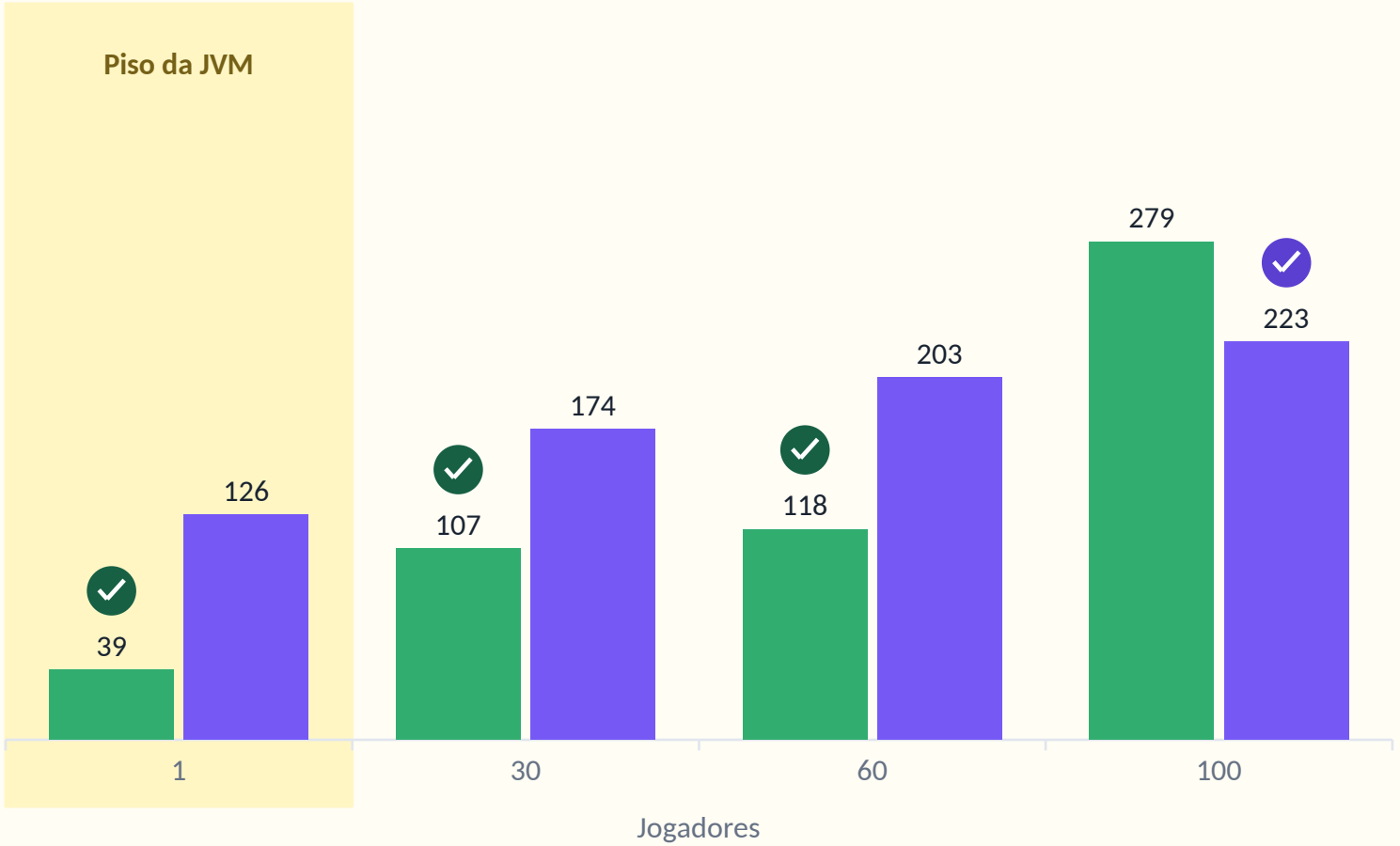
Fresh + Deno (SSR) | Angular + Spring (SPA) | ✓ vence nesta carga

Coleta de 12/10/2026 · 24 execuções · 3 pares por carga · mesma máquina

Memória: a JVM parte de um piso 3× maior

1–60 jog. **Fresh** | 100 jog. **Angular**

Memória do servidor, RSS (MiB) · *menor é melhor*



126 MiB

do Spring Boot com 1 jogador, contra 39 MiB do Deno

Até 60 jogadores o Deno usa menos memória. Em 100, no limite de CPU, passa a JVM: 279 contra 223 MiB.

RSS
RAM que o processo ocupa. Ex.: 126 MiB com o Spring quase parado.

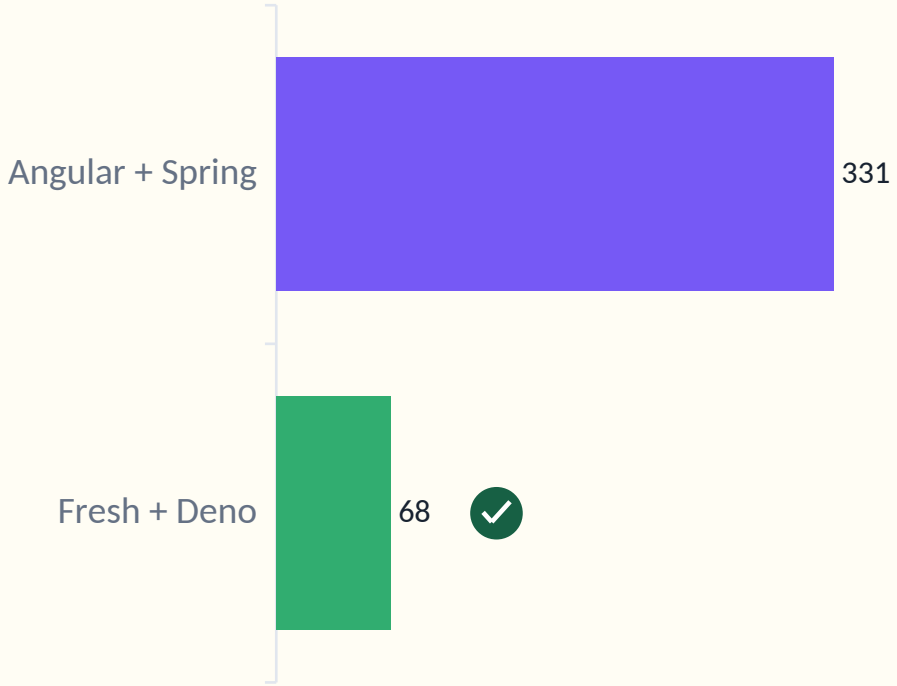
Fresh + Deno (SSR) | Angular + Spring (SPA) | ✓ vence nesta carga

Coleta de 12/10/2026 · 24 execuções · 3 pares por carga · mesma máquina

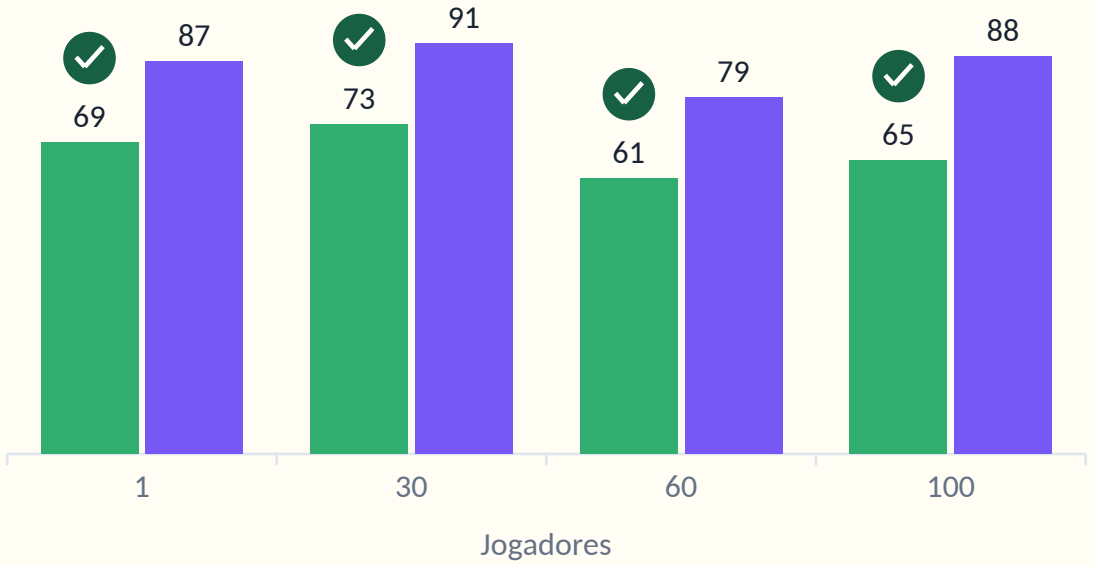
Cliente: a SPA baixa 5× mais bytes por página

Vence
Fresh + Deno

Bytes no carregamento inicial (KiB) · *menor é melhor*



Carregamento do jogador (ms, mediana) · *menor é melhor*



Jogador: Fresh 18–23 ms mais rápido em todas as cargas. Host: sem vencedor claro (155–198 ms nas duas).
Mediana: metade dos carregamentos foi mais rápida que o valor mostrado.

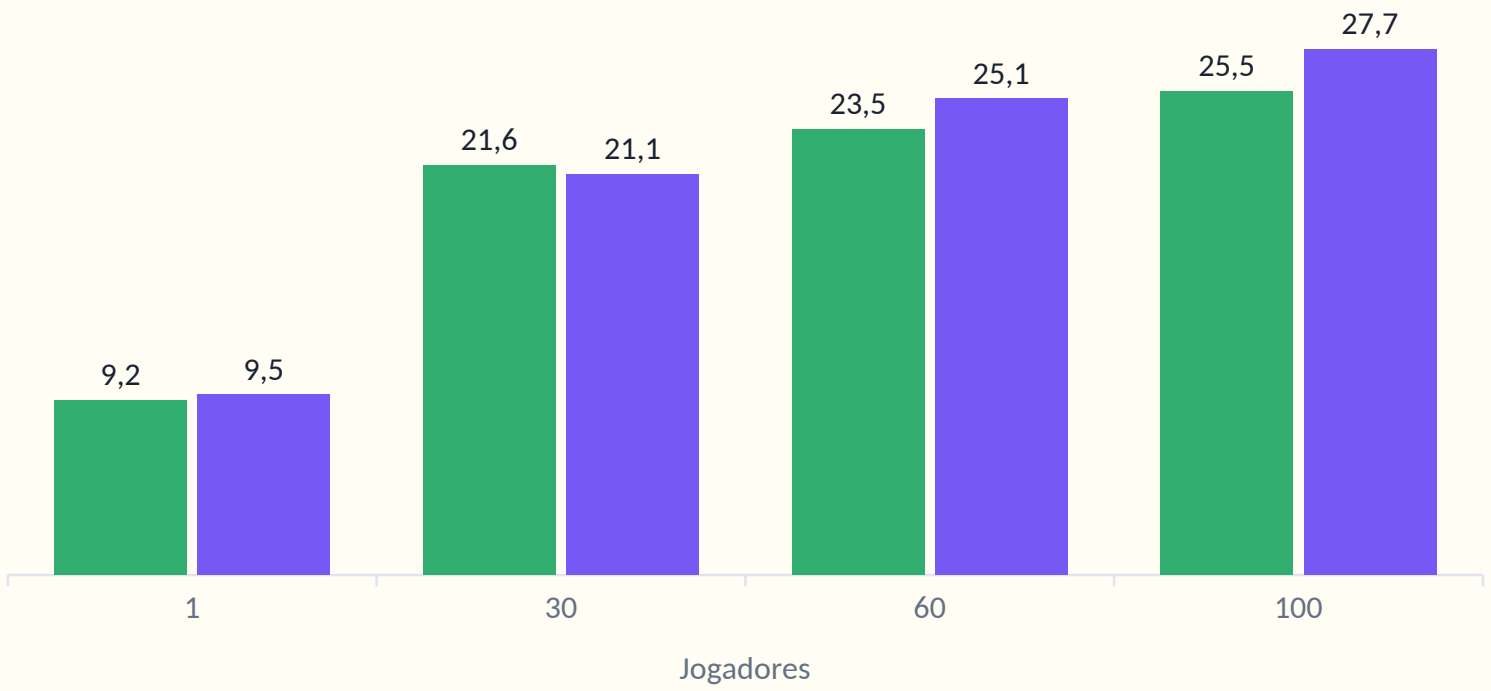
Fresh + Deno (SSR) · Angular + Spring (SPA) · vence nesta carga

Coleta de 12/10/2026 · 24 execuções · 3 pares por carga · mesma máquina

Navegador: custo praticamente igual nas duas

= Todas as cargas **Empate**

CPU média do Chromium, host + 1 jogador (% de 1 núcleo) · *menor é melhor*



Fresh + Deno (SSR) Angular + Spring (SPA)

≤ 2,2

pontos de CPU de diferença em qualquer carga

Memória também próxima: 563 a 628 MiB no Fresh, 563 a 668 MiB no Angular. Só 2 navegadores num desktop.

Como ler a CPU %
27,7% = pouco mais de 1/4 de um núcleo ocupado, em média.

Coleta de 12/10/2026 · 24 execuções · 3 pares por carga · mesma máquina

SSR ou SPA não decidiu o resultado

1

Por quê

Só 2 páginas são renderizadas por execução. O resto é API e SSE, igual nas duas.

2

O gargalo

Com 100 jogadores o único processo Deno ficou no limite de 1 núcleo e os pedidos fizeram fila.

3

Próximos testes

deno serve --parallel com 100 jogadores; throttling de CPU e rede nos navegadores.

Resultado exploratório: 3 pares por carga, mesma máquina; 100 jogadores ≠ 100 navegadores.